Samuel Gonzalez

Morgan Castle

Jayshawn Mercedes

DSC 623 - Project - Conceptual Design

Dr. Vanessa Aguiar-Pulido

March 24, 2026

A. Main Entities:
    a. Outlet
        i. The outlet represents a physical location of the company where vehicles are stored and staff members work. The system must keep track of each outlet in order to track their vehicles and employee locations.
    b. Client
        i. Clients are the individuals who rent the vehicles. The system must maintain records of the clients so that the hire agreements are associated with the right individuals.
    c. Vehicle
        i. Vehicles are the primary asset of the company. Each vehicle is uniquely identified via its registration number, allowing the system to store and monitor important information about each vehicle.
    d. Staff
        i. The staff represent the employees that work for the company. The system needs to store information about each employee and identify the outlet they work in.
    e. Hire Agreement
        i. Each Client signs a hiring agreement with the Company upon renting a vehicle. This allows the Company to keep track of who has one of their vehicles at any given time.

B. Entity Relationship(s)
    a. Outlet - stores - Vehicle
    b. Outlet - employs - Staff
    c. Client - signs - Hire Agreement
    d. Vehicle - assigned - Hire Agreement

C. Multiplicity Constraints
    a. Outlet to Vehicle = 1:*
        i. An outlet can store many vehicles
        ii. Each vehicle is stored at one outlet at a time
    b. Outlet to Staff = 1:*
        i. An outlet can employ many staff members
        ii. An employee can only work at one outlet
    c. Client to Hire Agreement = 1:*
        i. A client can sign many hire agreements
        ii. Each hire agreement belongs to one client
    d. Hire Agreement to Vehicle = *:1
        i. A vehicle can appear in many hire agreements

ii. Each hire agreement agreement involves one vehicle

Everything that was explained above is represented visually in this chart

| E1 | M1 | Relation | M2 | E2 | C |
|---|---|---|---|---|---|
| Outlet | 1..1 | Stores | 1..* | Vehicle | 1..* |
| Outlet | 1..1 | Employs | 1..* | Staff | 1..* |
| Client | 1..1 | Signs | 1..* | Hire Agreement | 1..* |
| Vehicle | 1..1 | Assigned | 1..* | Hire Agreement | 1..* |

D. Entity Attributes

a. Outlet
   i. OutletID (PK)
   ii. Address
   iii. PhoneNumber
   iv. FaxNumber

b. Client
   i. ClientID (PK)
   ii. FirstName
   iii. LastName
   iv. Address
   v. PhoneNumber
   vi. DateOfBirth
   vii. DrivingLicenseNumber

c. Vehicle
   i. RegistrationNumber (PK)
   ii. Model
   iii. Make
   iv. EngineSize
   v. Capacity
   vi. CurrentMileage
   vii. DailyHireRate
   viii. CurrentLocation

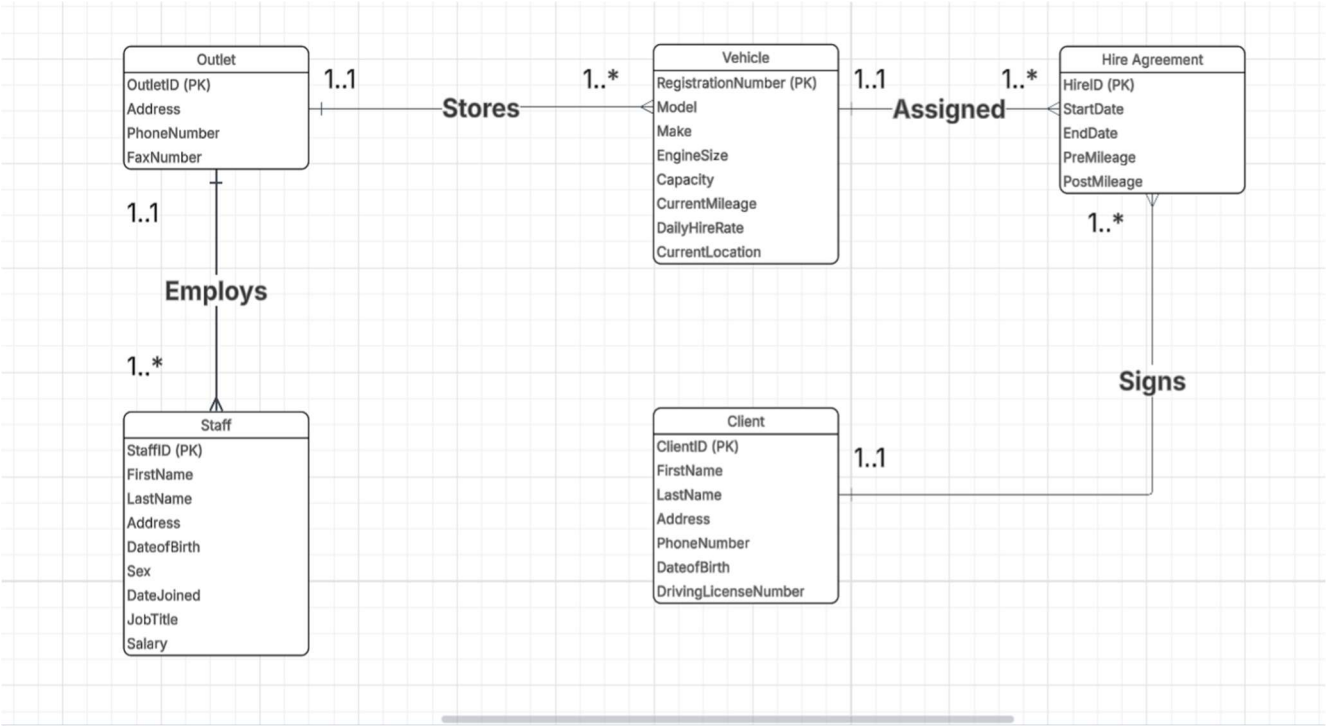
d. Staff
   i. StaffID (PK)
   ii. FirstName

iii. LastName
iv. Address
v. PhoneNumber
vi. DateOfBirth
vii. Sex
viii. DateJoined
ix. JobTitle
x. Salary

e. Hire Agreement
i. HireID (PK)
ii. StartDate
iii. EndDate
iv. PreMileage
v. PostMileage

E. Candidate and Primary Keys
a. Outlet
i. Outlet ID (PK)
ii. Candidate Keys: Outlet ID, address, phone number, Fax Number (assuming each branch has a unique address, phone number, and Fax Number)
b. Client
i. Client ID (PK)
ii. Candidate Keys: Client ID, Driving License Number
c. Vehicle
i. RegistrationNumber (PK)
ii. Candidate Keys: Registration Number
d. Staff
i. StaffID (PK)
ii. Candidate Keys: StaffID
e. Hire Agreement
i. Hire ID (PK)
ii. Candidate Keys: Hire ID

F. E-R Diagram


Outlet
OutletID (PK)
Address
PhoneNumber
FaxNumber
1..1
Stores
1..*
Vehicle
RegistrationNumber (PK)
Model
Make
EngineSize
Capacity
CurrentMileage
DailyHireRate
CurrentLocation
1..1
Assigned
1..*
Hire Agreement
HireID (PK)
StartDate
EndDate
PreMileage
PostMileage
1..1
Employs
1..*
1..*
Signs
Staff
StaffID (PK)
FirstName
LastName
Address
DateofBirth
Sex
DateJoined
JobTitle
Salary
Client
ClientID (PK)
FirstName
LastName
Address
PhoneNumber
DateofBirth
DrivingLicenseNumber
1..1

Samuel Gonzalez

Morgan Castle

Jayshawn Mercedes

DSC 623 - Project - Logical Design

Dr. Vanessa Aguiar-Pulido

April 9, 2026

A. Foreign Keys

a. Outlet(<u>OutletID</u> , Address, PhoneNumber, FaxNumber)
b. Vehicle(<u>RegistrationNumber</u>, OutletID (FK), Model, Make, EngineSize, Capacity, CurrentMileage, DailyHireRate, CurrentLocation)
c. Client(<u>ClientID</u>, FirstName, LastName, Address, PhoneNumber, DateOfBirth, DrivingLicenseNumber)
d. Staff(<u>StaffID</u>, OutletID (FK), FirstName, LastName, Address, PhoneNumber, DateOfBirth, Sex, DateJoined, JobTitle, Salary)
e. HireAgreement(<u>HireID</u>, RegistrationNumber (FK), ClientID (FK), StartDate, EndDate, PreMileage, PostMileage)

B. Logical Model Normalization - 3NF

- 1NF - All of the tables have non-repeating values, no multivalued attributes, each table has one primary key, and all values are atomic. Thus, our database is in 1NF.
- 2NF - As there are no composite primary keys, every non-key attribute is dependent on the whole primary key. Thus, there are no partial dependencies. So, the database is in 2NF.
- 3NF - All attributes depend on the primary key and no other attribute, so all of the tables are in 3NF.

C. Create five queries involving more than one entity (explain steps)

1. Retrieve all the staff members together with the outlet to which they are assigned, including the outlet number and staff details.
    a. Entities Involved:
        i. Staff
        ii. Outlet
    b. Steps to get result:
        i. Read Staff relation to get details of each staff member.
        ii. Read the Outlet relation to get outlet information.
        iii. Join Staff and Outlet using <u>OutletID</u>, since <u>OutletID</u> in Staff is a foreign key referencing Outlet.
        iv. Return the staff details along with the corresponding outlet information.

2. Get all the clients who have made more than one hire agreement, together with the vehicles they hired.
    a. Entities Involved:
        i. Client
        ii. HireAgreement
        iii. Vehicle
    b. Steps to get result:

i. Read the HireAgreement relation.
ii. Group the records by ClientID.
iii. Count the number of hire agreements for each client.
iv. Keep only those clients whose count is greater than one.
v. Join the filtered results with the Client relation using ClientID to get client details.
vi. Join the result with Vehicle relation using RegistrationNumber to get the details of the vehicles hired.
vii. Return the client details together with the corresponding hired vehicles.

3. Retrieve the vehicle that has been hired most frequently, based on the number of hire agreements associated with it.
   a. Entities Involved:
      i. Vehicle
      ii. HireAgreement
   b. Steps to get result:
      i. Read the HireAgreement relation.
      ii. Group the records by the RegistrationNumber.
      iii. Count the number of hire agreements associated with each vehicle.
      iv. Sort the grouped results in descending order of the hire count.
      v. Select the vehicle with the highest count.
      vi. Join the result with the Vehicle relation using the RegistrationNumber to display the full vehicle details.
      vii. Return the vehicle and its total number of hires.

4. Retrieve the hire history of a specific vehicle, including the client who hired it and the start and end dates of each hire agreement.
   a. Entities Involved:
      i. Vehicle
      ii. HireAgreement
      iii. Client
   b. Steps to get result:
      i. Read the Vehicle relation and identify the required vehicle using its RegistrationNumber.
      ii. Read the HireAgreement relation and select all the hire agreements associated with that RegistrationNumber.
      iii. Join the filtered hire agreements with the Client relation using ClientID to get client details.
      iv. Return the vehicle, the client who hired it, and the corresponding hire start and end dates.

5. Get all the vehicles currently assigned to each outlet, including the vehicle registration number, make, and model.
    a. Entities Involved:
        i. Vehicle
        ii. Outlet
    b. Steps to get result:
        i. Read the Vehicle relation to get the vehicle details.
        ii. Read the Outlet relation to get the outlet details.
        iii. Join Vehicle and Outlet using OutletID, since each vehicle is assigned to one outlet.
        iv. Return the outlet details together with the registration number, make, and model of each vehicle assigned to that outlet.

D. Define Integrity Constraints

I. Primary Key constraints:
    A. Outlet: outletNo
    B. Vehicle: registrationNo
    C. Client: clientNo
    D. Hire Agreement: hireID
    E. Staff: staffNo

*These attributes are unique and cannot be null

II. Referential Integrity/Foreign Key constraints:
    A. Vehicle.outletNo references Outlet.outletNo
    B. HireAgreement.clientNo references Client.clientNo
    C. HireAgreement.registrationNo references Vehicle.registrationNo
    D. Staff.outletNo references Outlet.outletNo

III. Alternate Key constraints:
    A. Client: Client.DrivingLicenseNumber
        1. The attribute is unique but cannot be null if hiring a vehicle

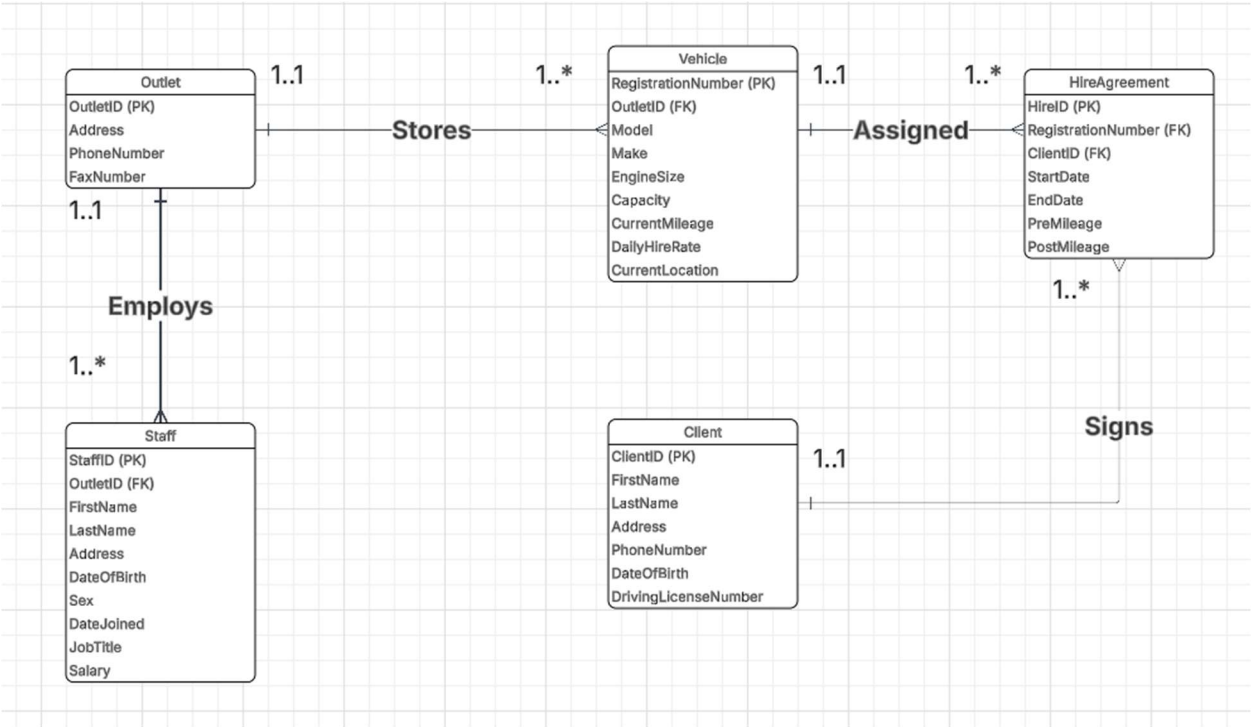
IV. Required Data:
    A. Outlet: OutletID, Address, PhoneNumber
    B. Vehicle: RegistrationNumber, OutletID, Model, Make, EngineSize, Capacity, CurrentMileage, DailyHireRate
    C. Client: ClientID, FirstName, LastName, Address, PhoneNumber, DateOfBirth, DrivingLicenseNumber

D. Staff: StaffID, OutletID, FirstName, LastName, Address, PhoneNumber, DateOfBirth, Sex, DateJoined, JobTitle, Salary
E. HireAgreement: HireID, RegistrationNumber, ClientID, StartDate, EndDate, PreMileage

V. Attribute Domain constraints
   A. EngineSize > 0
   B. Capacity must be a positive integer
   C. CurrentMileage, PreMileage, PostMileage >= 0
   D. DailyHireRate and Salary > 0
   E. Sex must take one of the following values: ‘M’, ‘F’, or ‘Other’
   F. DateOfBirth DateJoined, StartDate, and EndDate must contain valid date values
   G. RegistrationNumber must follow a valid alphanumeric registration format
   H. PhoneNumber must follow a valid phone number format
   I. DrivingLincenseNumber must be unique and follow a valid alphanumeric format

VI. General constraints:
   A. EndDate >= StartDate
   B. PostMileage >= PreMileage
   C. Clients must meet legal driving age based on their date of birth
   D. Each vehicle belongs to one outlet at a time
   E. Each staff member must belong to only one outlet
   F. Each hire agreement must be associated to exactly one client and one vehicle
   G. A vehicle cannot have overlapping hire periods; it can only be associated with the one active hire agreement at a time
   H. Salary and DailyHireRate > 0

E. E-R diagram


Outlet
OutletID (PK)
Address
PhoneNumber
FaxNumber
1..1
Stores
1..*
Vehicle
RegistrationNumber (PK)
OutletID (FK)
Model
Make
EngineSize
Capacity
CurrentMileage
DailyHireRate
CurrentLocation
1..1
Assigned
1..*
HireAgreement
HireID (PK)
RegistrationNumber (FK)
ClientID (FK)
StartDate
EndDate
PreMileage
PostMileage
1..1
Employs
1..*
1..*
Signs
Staff
StaffID (PK)
OutletID (FK)
FirstName
LastName
Address
DateOfBirth
Sex
DateJoined
JobTitle
Salary
Client
ClientID (PK)
FirstName
LastName
Address
PhoneNumber
DateOfBirth
DrivingLicenseNumber
1..1

Samuel Gonzalez

Morgan Castle

Jayshawn Mercedes

DSC 623 - Project - Implementation

Dr. Vanessa Aguiar-Pulido

April 30, 2026

A. SQL code used to Create Database Schema

a. Outlet relation

i. This table stores information about each rental outlet. OutletID is the primary key, meaning each outlet has a unique identifier. The table also stores the outlet's address, phone number, and fax number

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Outlet (
    OutletID INTEGER PRIMARY KEY,
    Address TEXT,
    PhoneNumber INTEGER,
    FaxNumber INTEGER
)
""")
```

b. Vehicle relation

i. This table stores details about each vehicle available for hire. RegistrationNumber is the primary key because each vehicle has a unique registration number. OutletID is a foreign key referencing the Outlet relation, which shows the outlet the vehicle is assigned to. Check constraints are used to make sure the mileage is not negative and that values such as the daily hire rate, engine size, and capacity are greater than zero.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Vehicle (
    RegistrationNumber TEXT PRIMARY KEY,
    OutletID INTEGER NOT NULL,
    Model TEXT,
    Make TEXT,
    EngineSize REAL,
    Capacity REAL,
    CurrentMileage INTEGER,
    DailyHireRate REAL,
    CurrentLocation TEXT,
    CHECK (CurrentMileage >= 0),
    CHECK (DailyHireRate > 0),
    CHECK (EngineSize > 0),
    CHECK (Capacity > 0),
    FOREIGN KEY (OutletID) REFERENCES Outlet(OutletID)
)
""")
```

c. Client relation

i. This table stores information about the customers that hire vehicles. ClientID is the primary key, meaning each client has a unique identifier. The relation also stores the client's name (first and last), address, phone

number, date of birth, and driving license number. The DrivingLicenseNumber field is marked as UNIQUE to make sure no two clients have the same license number.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Client (
    ClientID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    Address TEXT,
    PhoneNumber INTEGER,
    DateOfBirth TEXT,
    DrivingLicenseNumber TEXT UNIQUE
)
""")
```

d. Staff relation

   i. This table stores information about the employees working at the rental outlets. StaffID is the primary key, and OutletID is a foreign key that links each staff member to the outlet where they work. The relation also stores personal and job-related information such as name (first and last), address, phone number, date of birth, sex, date joined, job title, and salary. Check constraints are used to make sure the salary is greater than zero, and that the sex value is limited to ‘M’, ‘F’, or ‘Other’.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Staff (
    StaffID INTEGER PRIMARY KEY,
    OutletID INTEGER NOT NULL,
    FirstName TEXT,
    LastName TEXT,
    Address TEXT,
    PhoneNumber INTEGER,
    DateOfBirth TEXT,
    Sex TEXT CHECK (Sex IN ('M', 'F', 'Other')),
    DateJoined TEXT,
    JobTitle TEXT,
    Salary REAL,
    CHECK (Salary > 0),
    FOREIGN KEY (OutletID) REFERENCES Outlet(OutletID)
)
""")
```

e. HireAgreement relation

i. This table stores information about the vehicle rental agreements. HireID is the primary key, meaning each hire agreement has a unique identifier. RegistrationNumber is a foreign key referencing the Vehicle relation, and ClientID is a foreign key referencing the Client relation, This connects each agreement to the vehicle being hired and the client hiring it. Check constraints are used to make sure the end date is not before the start date, mileage values are not negative, and the post-hire mileage is greater than or equal to the pre-hire mileage.

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS HireAgreement (
    HireID INTEGER PRIMARY KEY,
    RegistrationNumber TEXT NOT NULL,
    ClientID INTEGER NOT NULL,
    StartDate TEXT,
    EndDate TEXT,
    PreMileage REAL,
    PostMileage REAL,
    CHECK (EndDate >= StartDate),
    CHECK (PreMileage >= 0),
    CHECK (PostMileage >= 0),
    CHECK (PostMileage >= PreMileage),
    FOREIGN KEY (RegistrationNumber) REFERENCES Vehicle(RegistrationNumber),
    FOREIGN KEY (ClientID) REFERENCES Client(ClientID)
)
""")
```

ii. Age Requirement Trigger

1. This enforces the rule that a client must be at least 16 years old before in order to rent a vehicle. Before a new record is inserted into this relation, SQLite checks the client's DateOfBirth from the Client table and then calculates the client's current age. If the client is younger than 16, a new record can not be created and an error message is returned.

```
cursor.execute("""
CREATE TRIGGER check_client_age
BEFORE INSERT ON HireAgreement
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Client does not meet the minimum driving age of 16')
    WHERE (
        SELECT (strftime('%Y', 'now') - strftime('%Y', DateOfBirth)
              - (strftime('%m-%d', 'now') < strftime('%m-%d', DateOfBirth)))
        FROM Client WHERE ClientID = NEW.ClientID
    ) < 16;
END;
""")
```

iii. No Overlapping Hire Dates Trigger

1. This prevents the same vehicle from being hired for overlapping time periods. Before inserting a new hire agreement, SQLite checks whether another hire agreement already exists for the same RegistrationNumber during the requested date range. If an overlap is found, the insert is stopped and an error message is returned.

```
cursor.execute("""
CREATE TRIGGER check_no_overlapping_hires
BEFORE INSERT ON HireAgreement
FOR EACH ROW
BEGIN
    SELECT RAISE(ABORT, 'Vehicle already has an active hire agreement during this period')
    WHERE EXISTS (
        SELECT 1 FROM HireAgreement
        WHERE RegistrationNumber = NEW.RegistrationNumber
        AND NOT (NEW.EndDate <= StartDate OR NEW.StartDate >= EndDate)
    );
END;
""")
```

B. Relation Tuples

a. Outlet tuples

i. Five sample records were inserted into the Outlet table to represent different rental locations across Florida. Each tuple includes a unique OutletID, along with the outlet's address, phone number, and fax number. This information is used to support relationships with staff and vehicles.

```
cursor.execute("""
INSERT INTO Outlet (OutletID, Address, PhoneNumber, FaxNumber) VALUES
(1, '123 Main St, Miami FL', 3051234567, 3051234568),
(2, '456 Oak Ave, Orlando FL', 4071234567, 4071234568),
(3, '789 Pine Rd, Tampa FL', 8131234567, 8131234568),
(4, '321 Elm St, Jacksonville FL', 9041234567, 9041234568),
(5, '654 Maple Dr, Tallahassee FL', 8501234567, 8501234568)
""")
```

```
# display Outlet relation
cursor.execute("""
SELECT * FROM Outlet;
""")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, '123 Main St, Miami FL', 3051234567, 3051234568)
(2, '456 Oak Ave, Orlando FL', 4071234567, 4071234568)
(3, '789 Pine Rd, Tampa FL', 8131234567, 8131234568)
(4, '321 Elm St, Jacksonville FL', 9041234567, 9041234568)
(5, '654 Maple Dr, Tallahassee FL', 8501234567, 8501234568)
```

b. Vehicle tuples

i. Five sample vehicles were inserted into the Vehicle relation. Each vehicle is identified by a unique RegistrationNumber and is assigned to an outlet through OutletID. The records include details such as make, model, engine size, capacity, mileage, daily hire rate, and current location.

```
cursor.execute("""
INSERT INTO Vehicle (RegistrationNumber, OutletID, Model, Make, EngineSize, Capacity, CurrentMileage, DailyHireRate, CurrentLocation) VALUES
('ABC123', 1, 'Camry', 'Toyota', 2.5, 5.0, 15000, 49.99, 'Miami FL'),
('XYZ789', 2, 'Civic', 'Honda', 1.8, 5.0, 22000, 39.99, 'Orlando FL'),
('DEF456', 3, 'Mustang', 'Ford', 5.0, 4.0, 8000, 89.99, 'Tampa FL'),
('GHI101', 4, 'Altima', 'Nissan', 2.0, 5.0, 31000, 44.99, 'Jacksonville FL'),
('JKL202', 5, 'Malibu', 'Chevrolet', 1.5, 5.0, 12000, 34.99, 'Tallahassee FL')
""")
```

```python
# display Vehicle relation
cursor.execute("""
SELECT * FROM Vehicle;
""")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
('ABC123', 1, 'Camry', 'Toyota', 2.5, 5.0, 15000, 49.99, 'Miami FL')
('XYZ789', 2, 'Civic', 'Honda', 1.8, 5.0, 22000, 39.99, 'Orlando FL')
('DEF456', 3, 'Mustang', 'Ford', 5.0, 4.0, 8000, 89.99, 'Tampa FL')
('GHI101', 4, 'Altima', 'Nissan', 2.0, 5.0, 31000, 44.99, 'Jacksonville FL')
('JKL202', 5, 'Malibu', 'Chevrolet', 1.5, 5.0, 12000, 34.99, 'Tallahassee FL')
```

c. Client tuples

i. Five client records were inserted into the Client relation. Each client has a unique ClientID and a unique DrivingLicenseNumber. The table stores each of the clients personal data such as name (first and last), address, phone number, and date of birth, which is later used for enforcing business rules like minimum driving age.

```python
cursor.execute("""
INSERT INTO Client (ClientID, FirstName, LastName, Address, PhoneNumber, DateOfBirth, DrivingLicenseNumber) VALUES
(1, 'John', 'Smith', '10 River Rd, Miami FL', 3059876543, '1990-03-15', 'DL123456'),
(2, 'Maria', 'Garcia', '20 Lake Ave, Orlando FL', 4079876543, '1985-07-22', 'DL789012'),
(3, 'James', 'Brown', '30 Hill St, Tampa FL', 8139876543, '1992-11-05', 'DL345678'),
(4, 'Ashley', 'Jones', '40 Bay Dr, Jacksonville FL', 9049876543, '1998-01-30', 'DL901234'),
(5, 'Carlos', 'Rivera', '50 Coast Blvd, Tallahassee FL', 8509876543, '2000-06-18', 'DL567890')
""")
```

```python
# display Client relation
cursor.execute("""
SELECT * FROM Client;
""")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, 'John', 'Smith', '10 River Rd, Miami FL', 3059876543, '1990-03-15', 'DL123456')
(2, 'Maria', 'Garcia', '20 Lake Ave, Orlando FL', 4079876543, '1985-07-22', 'DL789012')
(3, 'James', 'Brown', '30 Hill St, Tampa FL', 8139876543, '1992-11-05', 'DL345678')
(4, 'Ashley', 'Jones', '40 Bay Dr, Jacksonville FL', 9049876543, '1998-01-30', 'DL901234')
(5, 'Carlos', 'Rivera', '50 Coast Blvd, Tallahassee FL', 8509876543, '2000-06-18', 'DL567890')
```

d. Staff tuples

i. Five staff members were created, with each staff member being assigned to an outed using OutletID, establishing a relationship between staff and outlet locations. The tuples also include personal details, job roles, date joined, and salary information.

```python
cursor.execute("""
INSERT INTO Staff (StaffID, OutletID, FirstName, LastName, Address, PhoneNumber, DateOfBirth, Sex, DateJoined, JobTitle, Salary) VALUES
(1, 1, 'Linda', 'White', '11 Palm St, Miami FL', 3051112222, '1980-04-10', 'F', '2015-06-01', 'Manager', 55000.00),
(2, 2, 'Robert', 'Lee', '22 Oak Ave, Orlando FL', 4071112222, '1975-09-25', 'M', '2010-03-15', 'Supervisor', 48000.00),
(3, 3, 'Sarah', 'Kim', '33 Pine Rd, Tampa FL', 8131112222, '1990-12-01', 'F', '2018-08-20', 'Agent', 38000.00),
(4, 4, 'David', 'Chen', '44 Elm St, Jacksonville FL', 9041112222, '1988-02-14', 'M', '2017-01-10', 'Agent', 37000.00),
(5, 5, 'Emily', 'Davis', '55 Maple Dr, Tallahassee FL', 8501112222, '1995-07-07', 'F', '2020-05-05', 'Agent', 36000.00)
""")
```

```python
# display Staff relation
cursor.execute("""
SELECT * FROM Staff;
""")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, 1, 'Linda', 'White', '11 Palm St, Miami FL', 3051112222, '1980-04-10', 'F', '2015-06-01', 'Manager', 55000.0)
(2, 2, 'Robert', 'Lee', '22 Oak Ave, Orlando FL', 4071112222, '1975-09-25', 'M', '2010-03-15', 'Supervisor', 48000.0)
(3, 3, 'Sarah', 'Kim', '33 Pine Rd, Tampa FL', 8131112222, '1990-12-01', 'F', '2018-08-20', 'Agent', 38000.0)
(4, 4, 'David', 'Chen', '44 Elm St, Jacksonville FL', 9041112222, '1988-02-14', 'M', '2017-01-10', 'Agent', 37000.0)
(5, 5, 'Emily', 'Davis', '55 Maple Dr, Tallahassee FL', 8501112222, '1995-07-07', 'F', '2020-05-05', 'Agent', 36000.0)
```

e. HireAgreement tuples

i. Five hire agreements were inserted into the HireAgreement relation. Each record links a client vehicle using ClientID and RegistrationNumber. The table stores the start and end dates of the hire, along with mileage before and after the rental, allowing tracking of vehicle usage over time.

```python
cursor.execute("""
INSERT INTO HireAgreement (HireID, RegistrationNumber, ClientID, StartDate, EndDate, PreMileage, PostMileage) VALUES
(1, 'ABC123', 1, '2026-01-01', '2026-01-05', 15000, 15300),
(2, 'XYZ789', 2, '2026-01-10', '2026-01-15', 22000, 22400),
(3, 'DEF456', 3, '2026-02-01', '2026-02-03', 8000, 8150),
(4, 'GHI101', 4, '2026-02-20', '2026-02-25', 31000, 31500),
(5, 'JKL202', 5, '2026-03-01', '2026-03-07', 12000, 12600)
""")
```

```python
# display HireAgreement relation
cursor.execute("""
SELECT * FROM HireAgreement;
""")
rows = cursor.fetchall()
for row in rows:
    print(row)
```

```
(1, 'ABC123', 1, '2026-01-01', '2026-01-05', 15000.0, 15300.0)
(2, 'XYZ789', 2, '2026-01-10', '2026-01-15', 22000.0, 22400.0)
(3, 'DEF456', 3, '2026-02-01', '2026-02-03', 8000.0, 8150.0)
(4, 'GHI101', 4, '2026-02-20', '2026-02-25', 31000.0, 31500.0)
(5, 'JKL202', 5, '2026-03-01', '2026-03-07', 12000.0, 12600.0)
```

C. SQL Queries

a. Query #1

```
cursor.execute("""
SELECT
    S.StaffID,
    S.FirstName,
    S.LastName,
    S.JobTitle,
    S.Salary,
    O.OutletID,
    O.Address,
    O.PhoneNumber
FROM Staff S
JOIN Outlet O
    ON S.OutletID = O.OutletID;
""")
```

i. This query joins the Staff and Outlet tables using OutletID to display each staff member along with the outlet where they work. It verifies the relationship between staff and outlets.

```
(1, 'Linda', 'White', 'Manager', 55000.0, 1, '123 Main St, Miami FL', 3051234567)
(2, 'Robert', 'Lee', 'Supervisor', 48000.0, 2, '456 Oak Ave, Orlando FL', 4071234567)
(3, 'Sarah', 'Kim', 'Agent', 38000.0, 3, '789 Pine Rd, Tampa FL', 8131234567)
(4, 'David', 'Chen', 'Agent', 37000.0, 4, '321 Elm St, Jacksonville FL', 9041234567)
(5, 'Emily', 'Davis', 'Agent', 36000.0, 5, '654 Maple Dr, Tallahassee FL', 8501234567)
```

b. Query #2

```
cursor.execute("""
SELECT
    C.ClientID,
    C.FirstName,
    C.LastName,
    C.PhoneNumber,
    HA.HireID,
    V.RegistrationNumber,
    V.Make,
    V.Model
FROM Client C
JOIN HireAgreement HA
    ON C.ClientID = HA.ClientID
JOIN Vehicle V
    ON HA.RegistrationNumber = V.RegistrationNumber
WHERE C.ClientID IN (
    SELECT ClientID
    FROM HireAgreement
    GROUP BY ClientID
    HAVING COUNT(*) > 1
);
""")
```

i. This query identifies clients who have more than one hire agreement by grouping records in the HireAgreement relation. It joins with Client and Vehicle to display the client details along with the vehicles they have hired.

ii. There is no output since no client has more than one hire agreement.

c. Query #3

```
cursor.execute("""
SELECT
    V.RegistrationNumber,
    V.Make,
    V.Model,
    V.EngineSize,
    V.Capacity,
    V.CurrentMileage,
    V.DailyHireRate,
    COUNT(HA.HireID) AS TotalHires
FROM Vehicle V
JOIN HireAgreement HA
    ON V.RegistrationNumber = HA.RegistrationNumber
GROUP BY
    V.RegistrationNumber,
    V.Make,
    V.Model,
    V.EngineSize,
    V.Capacity,
    V.CurrentMileage,
    V.DailyHireRate
ORDER BY TotalHires DESC
LIMIT 1;
""")
```

i. This query counts how many times each vehicle appears in the HireAgreement relation and orders the results in descending order. It returns the vehicle with the highest number of hires along with its details.

```
('ABC123', 'Toyota', 'Camry', 2.5, 5.0, 15000, 49.99, 1)
```

d. Query #4

```
cursor.execute("""
SELECT
    V.RegistrationNumber,
    V.Make,
    V.Model,
    C.ClientID,
    C.FirstName,
    C.LastName,
    HA.StartDate,
    HA.EndDate
FROM Vehicle V
JOIN HireAgreement HA
    ON V.RegistrationNumber = HA.RegistrationNumber
JOIN Client C
    ON HA.ClientID = C.ClientID
WHERE V.RegistrationNumber = 'GHI101';
""")
```

i. This query retrieves all the hire agreements for a specific vehicle by filtering on RegistrationNumber. It joins with the Client relation to show which client(s) hired the vehicle and the corresponding dates.

```
('GHI101', 'Nissan', 'Altima', 4, 'Ashley', 'Jones', '2026-02-20', '2026-02-25')
```

e. Query #5

```
cursor.execute("""
SELECT
    O.OutletID,
    O.Address,
    O.PhoneNumber,
    V.RegistrationNumber,
    V.Make,
    V.Model
FROM Outlet O
JOIN Vehicle V
    ON O.OutletID = V.OutletID
ORDER BY O.OutletID;
""")
```

i. This query joins the Vehicle and Outlet relations using OutletID to display all vehicles assigned to each outlet. It returns outlet details along with the registration number, make, and model of each vehicle.

```
(1, '123 Main St, Miami FL', 3051234567, 'ABC123', 'Toyota', 'Camry')
(2, '456 Oak Ave, Orlando FL', 4071234567, 'XYZ789', 'Honda', 'Civic')
(3, '789 Pine Rd, Tampa FL', 8131234567, 'DEF456', 'Ford', 'Mustang')
(4, '321 Elm St, Jacksonville FL', 9041234567, 'GHI101', 'Nissan', 'Altima')
(5, '654 Maple Dr, Tallahassee FL', 8501234567, 'JKL202', 'Chevrolet', 'Malibu')
```

D. Github Repository Link
[https://github.com/JeiOkami/DSC-623---Project.git]